



A novel FPGA-based garbled circuit accelerator for secure two-party multiplication (GCMult)

Hiva Assasi¹ · Shahram Etemadi Borujeni¹

Received: 2 July 2025 / Accepted: 27 July 2026
© The Author(s) 2026

Abstract

Privacy-preserving function execution is crucial for applications such as cloud-based machine learning and IoT. While garbled circuits offer a viable solution, they suffer from high computational and communication costs. Multiplication, a fundamental operation in computing, plays a critical role in overall efficiency. Although parallel execution of independent operations has been widely explored, no prior work has investigated the decomposition of a single multiplication operation into smaller concurrent sub-operations within the garbled circuit framework. This paper introduces GCMult (Garbled Circuit Multiplier), a novel accelerator for garbling secure two-party multiplication that leverages a parallelized decomposition technique for Boolean circuits. Our method applies the Karatsuba algorithm to enhance concurrency while significantly reducing the number of gates, particularly high-cost AND gates. The proposed technique enables a specialized accelerator for garbling multiplication circuits, delivering substantial improvements in both computational speed and communication efficiency. FPGA-based experimental results demonstrate an average 258% reduction in garbling time and more than a 60% improvement in communication bandwidth on average compared to the best existing hardware accelerator. Additionally, our approach achieves a 4140% speedup over the fastest known software implementation, showcasing its potential for scalable and efficient privacy-preserving computations.

Keywords Privacy-preserving computation · Secure multi-party computation · Garbled circuits · Parallelization · Multiplication

1 Introduction

For many applications such as cloud-based machine learning inference and IoT, it is essential that operations be jointly executed by multiple parties [1, 2]. When data privacy is critical, participating parties must not gain access to each other's private data. Yao's garbled circuits (GC) technique [3] is one of the most well-known methods for secure computation [4, 5]. Although Yao's garbled circuits method is promising, it suffers from significant cost limitations. Tackling these restrictions, researchers have proposed various approaches [6, 7].

Indeed, since garbled circuit theory and implementation technologies were still in their early stages and the computation was a major bottleneck [8, 9], different technologies were proposed to improve the efficiency of computations. According to Beaver et al. [10] and Malkhi et al. [11], a point-and-permute algorithm can reduce the number of decryption operations per gate from 4 to 1. Naor et al. [12] proposed the GRR3 optimization method in 1999 that forces the first row of the garbled table to 0. This reduces the size of the garbled table for each gate to three rows. Using this approach, the garbled table for circuit gates with two inputs can be reduced by 25%, and it can be compatible with the FreeXOR [13] technology. Pinkas et al. [14] reduced a garbled table to two rows using polynomial interpolation. Gates in a circuit can be classified as odd gates or even gates depending on whether the output column has an odd number of 1's or not. AND and OR gates have a truth table with one and three 1's respectively, so they are odd gates. The XOR truth table contains two 1's in the output column, meaning that XOR gates are classified as even gates. Rather than decrypting the output

✉ Hiva Assasi
assasi@eng.ui.ac.ir

Shahram Etemadi Borujeni
etemadi@eng.ui.ac.ir

¹ Department of Computer Architecture, Faculty of Computer Engineering, University of Isfahan, Isfahan, Iran

line, this method uses polynomial interpolation to determine its garbled key. Additionally, Innocent et al. [15] proposed a structure of garbled circuits using universal gates. NOR and NAND logic gates are universal gates. From the universal gates can be generated the other logic gates like AND, OR, XOR, and NOT gates. NOR gates optimization was focused on in their study. Their garbled table was reduced to two rows per gate with just two encryption/decryption calls. By doing this, the bit complexity per gate will be reduced by 33%.

Multiplication is one of the most frequently used operations in computational systems, which, if performed efficiently, can significantly increase the overall efficiency. However, the cost of secure two-party multiplication increases dramatically with the size of the multiplication. Reviewing past valuable research, to the best of our knowledge, none of the existing methods has focused on the parallelization opportunities that could be achieved for a specific design. On the other hand, multiplication has been one of the costliest (time and space) ubiquitous basic operations.

Several researchers [16–20] have proposed compilers that take high-level descriptions like C code as input and try to automatically parallelize the evaluation of the function. In this regard, they have presented general-purpose compilers that consider multiplication (if visited in the input code) as one basic operation and do not exploit the maximum degree of parallelism as we do by breaking down the intrinsic definition of multiplication.

Several recent studies [21–23] have explored the acceleration of garbled circuits (GC) using FPGAs. One notable effort is **GarbledCPU** [21], which introduces a secure MIPS processor where the netlist is always a Boolean circuit representing the processor itself, allowing users to load and execute secure functions in binary form. This approach provides flexibility in programming languages but comes at a significant cost—executing a large and complex netlist introduces substantial overhead. GarbledCPU offers three versions, each balancing speed and privacy, but even its least secure variant incurs excessive computational costs, making it impractical for real-world applications.

Another general-purpose FPGA-based GC accelerator is FASE, presented in [22]. While this work achieves notable improvements in garbling time and introduces a well-structured design that has inspired aspects of our implementation, it relies on a **sequential Boolean circuit execution model**. Consequently, the communication bandwidth requirement between the garbler (FPGA) and the evaluator (typically a host CPU) is directly tied to the number of clock cycles required for execution. When the data overhead becomes excessively large, it creates a communication bottleneck, limiting the accelerator's practical usability.

An additional generic GC accelerator is introduced in [23]. However, this design does not fully exploit the computational

capabilities of the underlying hardware due to two key limitations. First, it employs a **basic gate scheduling mechanism**, which leads to significant periods where encryption units remain idle, reducing overall efficiency. Second, it does not integrate **Half-Gate optimization**, a crucial advancement in garbled circuits that reduces the size of garbled tables for non-XOR gates. As a result, it generates significantly more data than necessary, further increasing communication and storage costs. Additionally, this accelerator relies on **SHA-1 for encryption**, which is no longer considered secure [24–26].

Another GC accelerator is proposed in [27], but it is implemented as an **ASIC**, which introduces inherent disadvantages. Unlike FPGAs, ASICs lack **flexibility and reconfigurability**, making them unsuitable for applications that require frequent updates or modifications. Additionally, ASIC-based solutions involve longer development cycles and higher costs, limiting their adaptability to evolving cryptographic needs.

Moreover, all the aforementioned accelerators are **generic, multi-purpose designs** rather than **specialized architectures for multiplication operations**. Since they do not have prior knowledge of the function's Boolean circuit, they require an externally supplied netlist, leading to multiple inefficiencies. These include **increased communication bandwidth overhead, delayed garbling process initiation, and additional control logic complexity**.

Based on the reviewed literature, no published work has proposed a technique for parallelizing the garbled-circuit implementation of secure two-party multiplication. In contrast, our work introduces a dedicated, optimized approach for secure two-party multiplication, addressing these limitations and achieving substantial improvements in both performance and efficiency. Our approach exploits the inherent parallelism in the multiplication process by modifying the architecture. In this way, we reduce the computational and communication costs of privacy-preserving execution of large multiplications.

In this regard, first, a brief review and introduction of Yao's garbled circuit and its most important optimizations that our approach is based upon, FreeXOR and Half-Gate, are presented (Sects. 2 and 3). Sections 4 and 5 present the multiplication structure and the theoretical foundations of the decomposition method used in this paper. Section 6 explains the implementation structure of the proposed accelerator, followed by evaluations and the results in Sect. 7. Section 8 delineates the target application domains of GCMult and describes its integration within existing secure computation frameworks. Finally, Sect. 9 concludes the paper.

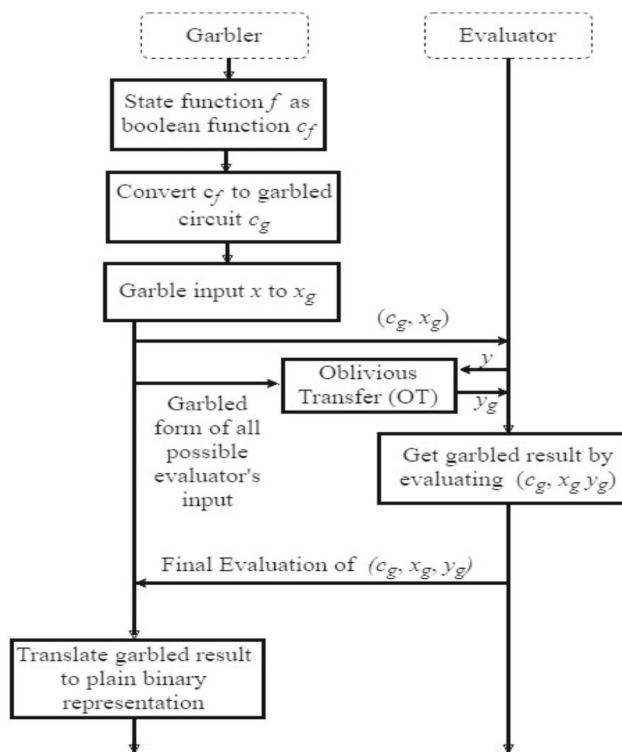


Fig. 1 Flowchart describing cooperation between garbler, and evaluator

2 Brief review of garbled circuits

An insecure circuit protocol involves two mutually untrusted parties: the circuit generator (garbler) and the circuit evaluator. Suppose the evaluator wishes to compute a value $z = f(x, y)$, where f is a function agreed upon by both parties. Neither the evaluator nor the garbler is willing to reveal any information about their private inputs beyond their sizes and what can be inferred from the output z . Therefore, both inputs must remain secret.

Figure 1 illustrates the steps involved in computing $z = f(x, y)$ in the basic garbled circuit evaluation protocol [28]. The left side of the flowchart represents the garbler's process, while the right side corresponds to the evaluator's actions. The garbler is responsible for garbling (encrypting) the Boolean circuit representing the function as well as encoding the input values. The evaluator, in turn, evaluates the garbled circuit on the garbled inputs received from the garbler. Below is a summary of each step in the flowchart.

2.1 Circuit construction

The procedure begins with the garbler, who constructs a garbled representation of the function to be evaluated on both her input and the evaluator's input:

1. **Circuit representation.** The garbler represents the function f as a Boolean circuit c_f . Each gate in the circuit has four rows in its truth table: two input wires (input columns) and one output wire (output column) in each row.
2. **Garbled circuit creation.** The garbler generates the garbled circuit c_g from c_f . She assigns two encryption keys to every wire—one for each possible wire value (0 or 1). Then, for each gate, she encrypts the output column of the truth table using the corresponding input-wire keys. The rows of the resulting garbled truth table are then randomly permuted.
3. **Garbling the garbler's input.** Based on c_g , the garbler garbles each bit of her input by encrypting it with the key associated with the corresponding input wire. This produces the garbled input x_g .
4. **Sending data to the evaluator.** The garbler sends c_g and x_g to the evaluator. The evaluator learns nothing about the garbler's private input because the structure of c_g does not reveal any such information.

2.2 Oblivious transfer

For each bit of the evaluator's input y , the evaluator obtains a corresponding garbled value y_g through the oblivious transfer (OT) protocol [30]. In each instance of the OT protocol, the garbler provides two keys (k_0, k_1) , and the evaluator provides a selection bit b . The protocol outputs the key k_b to the evaluator. The oblivious transfer of k_b guarantees that k_{1-b} remains hidden from the evaluator—ensuring that the function f cannot be evaluated on any input other than y . At the same time, the protocol ensures that the selection bit b remains hidden from the garbler.

2.3 Circuit evaluation

The evaluator decrypts each gate in c_g using x_g and y_g as the corresponding decryption keys. After evaluating all gates, the final output keys are mapped back to their corresponding bit values. In this protocol, the evaluator learns only the final output $z = f(x, y)$, since she cannot evaluate the circuit on any inputs other than x and y , and therefore cannot “decode” any intermediate values.

3 More on two important GC optimizations

Since the two previously mentioned optimizations for garbled circuits are central to our work, we provide a more detailed explanation of them here.

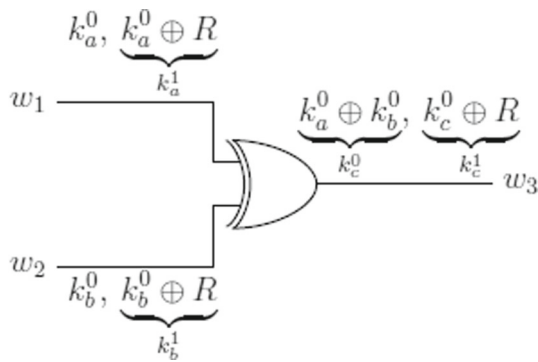


Fig. 2 XOR gates for free [29]

3.1 FreeXOR

Rather than assigning purely random labels to each wire, Kolesnikov and Schneider [13] proposed an alternative approach that introduces a structural relationship between the labels. Their key optimization enables XOR gates to be computed “for free”, meaning the garbler does not need to perform any encryption, and the evaluator does not need to perform any decryption. This is accomplished as follows. First, choose a random value R . As usual, assign the input labels k_a^0 randomly, meaning the label corresponding to 0 is chosen randomly for each input wire in every gate. However, instead of selecting k_a^1 at random, define it as $k_a^1 = k_a^0 \oplus R$. Next, determine the output label for 0 as $k_c^0 = k_a^0 \oplus k_b^0$. Visually, this can be represented as follows (Fig. 2):

A crucial benefit of this approach—central to optimizing GC implementations—is that during both garbling and evaluation, XORing the input labels (e.g., k_a^i and k_b^j) directly yields the correct output label. Thus, neither encryption by the garbler nor decryption by the evaluator is required; both simply XOR the input wire labels to compute the output wire label. The process is as follows:

$$k_a^0 \oplus k_b^0 = k_c^0 (\text{by definition})$$

$$k_a^0 \oplus k_b^1 = k_a^0 \oplus (k_b^0 \oplus R) = (k_a^0 \oplus k_b^0) \oplus R = k_c^0 \oplus R = k_c^1$$

$$k_a^1 \oplus k_b^0 = (k_a^0 \oplus R) \oplus k_b^0 = (k_a^0 \oplus k_b^0) \oplus R = k_c^0 \oplus R = k_c^1$$

$$\begin{aligned} k_a^1 \oplus k_b^1 &= (k_a^0 \oplus R) \oplus (k_b^0 \oplus R) \\ &= (k_a^0 \oplus k_b^0) \oplus (R \oplus R) \\ &= k_a^0 \oplus k_b^0 = k_c^0 \end{aligned}$$

The FreeXOR optimization encourages incorporating as many XOR gates as possible in the Boolean circuit designs intended for private execution using GCs [29].

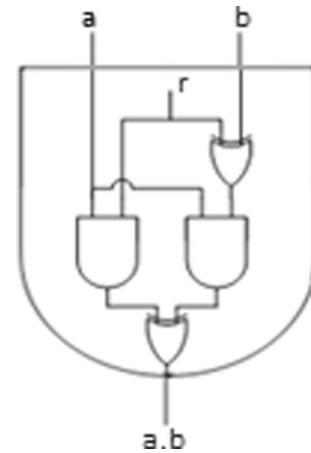


Fig. 3 Half-Gate AND

3.2 Half-Gate AND

Zahur et al. [31] introduced an optimization technique called Half-Gate for AND gates. This method enhances the efficiency of a GC’s AND gate by cutting the number of garbled table rows in half. It has been demonstrated as the most optimal approach for handling garbled tables in AND gates. The core concept of Half-Gate technology is depicted in Fig. 3. In this method, the input and output values of an AND gate are represented by a , b , and c , respectively. By introducing a random bit r , the AND gate expression can be reformulated as follows:

$$c = a \wedge b = a \wedge (r \oplus r \oplus b) = (a \wedge r) \oplus (a \wedge (r \oplus b))$$

The following algorithm describes the garbling procedure for Half-Gate [31]:

```

procedure GbAnd( $W_a^0, W_b^0$ ):
   $p_b \leftarrow \text{lsb } W_b^0$ 
   $j \leftarrow \text{nextIndex}(); j' \leftarrow \text{nextIndex}()$ 
  {First Half-Gate}
   $T_G \leftarrow H(W_a^0, j) + H(W_a^1, j) + p_b R$ 
  {Second Half-Gate}
   $T_E \leftarrow H(W_b^0, j') + H(W_b^1, j') + W_a^0$ 
  return ( $T_G, T_E$ )

```

Since AES is widely regarded as a secure encryption scheme, we use AES-128 for the hash function, Fig. 4 illustrates the implementation schematic:

As seen in both the algorithm and the schematic, garbling each AND gate requires four AES encryption operations, which significantly contribute to the overall time and hardware resource consumption in GC designs. (It is worth noting that the evaluation of the Half-Gate approach is similar but requires only two AES calls instead of four). Fortunately,

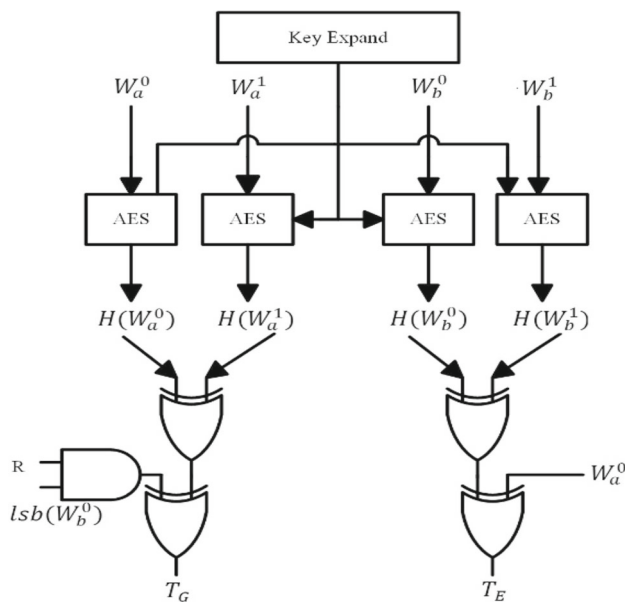


Fig. 4 Garbler's Half-Gate AND [27, 31]

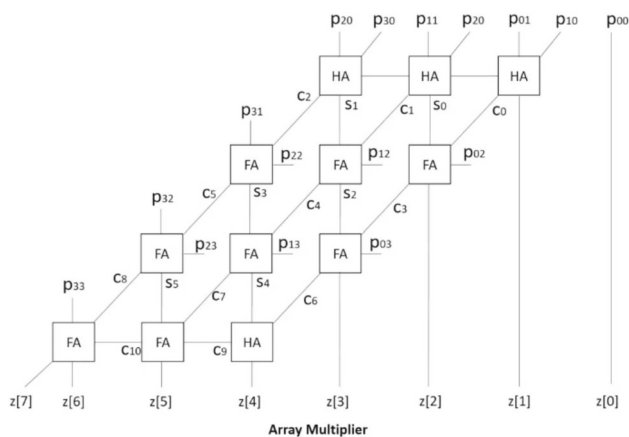


Fig. 5 A 4×4 array multiplier; example of the original structure used for multiplication Boolean circuit [32]

these operations are independent of one another and can therefore be executed in parallel.

4 Multiplication Boolean circuit

The multiplication structure used in our work, similar to previous studies [22, 23], is an $m \times n$ array multiplier [32]; an example of which is illustrated in Fig. 5. Providing a brief overview of this structure's key components clarifies the performance enhancements and FPGA resource optimization achieved by the proposed approach. As shown in the figure, the main components include full adders, half adders, and AND gates (represented as P_{ij} in the schematic).

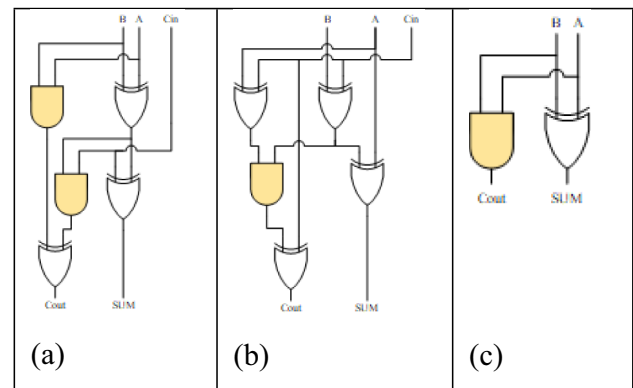


Fig. 6 **a** Typical full adder; extra AND could be substituted **b** full adder with one AND; proper for GCs **c** half adder; no extra AND

Table 1 Total number of gates in a $m \times n$ array multiplier

Gate	Number
AND	$m(2n - 1)$
XOR	$4mn - 4m - 3n$

Since the AND operation is the only one requiring time- and resource-intensive encryption (during garbling) or decryption (during evaluation), reducing the total number of AND operations significantly lowers the overall system processing cost. Therefore, instead of using the conventional full adder design shown in Fig. 6a, which includes two AND gates, we adopt the functionally equivalent structure in Fig. 6b, which contains only one.

While traditional references often describe all adders in an array multiplier as full adders, in the context of GCs, it is crucial to use half adders whenever possible. This is because, although XOR operations do not require encryption or decryption, they still consume clock cycles for processing (as explained in Section 6.2), and minimizing them helps improve overall efficiency. Unlike full adders, which contain four XOR operations, half adders, shown in Fig. 6c, have only one.

In an $m \times n$ array multiplier, there are n half adders, $mn - (m+n)$ full adders, and $m \times n$ AND gates. Table 1 summarizes the total number of AND and XOR operations (considering the full adder in Fig. 6b).

It is important to note that the values in Table 1 represent only the primary XOR operations within the array multiplier structure. The internal XOR operations of the Half-Gate AND (discussed in Section 3.2), along with the associated processing time and resource consumption, are already accounted for within the Half-Gate AND component.

5 Employed multiplication decomposition method

In this section, we will introduce a method to break multiplication into smaller (and hence, less costly) multiplications (which could be performed simultaneously, and we will benefit from that as parallelism), and some additions (which are much lower-cost operations). Then we will show how some optimizations could be done in the implementation of this method to use smaller addition operations in garbled circuit-based implementation.

The Karatsuba algorithm [33–35] has a complexity $n^{\log_2 3}$, and can be outlined as follows. Consider u and v to be two integers of n decimal digits each. Split each operand into roughly equal parts:

$$u = 2^s w + x, \quad v = 2^s y + z \quad (1)$$

where $0 \leq x < 2^s$, $0 \leq z < 2^s$, and $s = n/2$. The integers w and y therefore both have $n/2$ digits. Once that is done, the product becomes

$$uv = 2^{2s} wy + 2^s(wz + xy) + xz \quad (2)$$

The algorithm involves the introduction of an auxiliary variable

$$r = (w + x)(y + z) = wy + (wz + xy) + xz \quad (3)$$

This variable is used to rewrite the product as

$$uv = 2^{2s} wy + 2^s(r - wy - xz) + xz \quad (4)$$

Therefore, the computation of uv requires only three multiplications of two integers with $n/2$ digits. So, $M(u, v)$, which is a recursive method of multiplying large integers u and v , can be written as follows:

Step 1:

$$\begin{aligned} s &\leftarrow n \text{ div } 2; \quad w \leftarrow u \text{ div } 2^s; \quad x \\ &\leftarrow u \bmod 2^s; \quad y \leftarrow v \text{ div } 2^s; \quad z \leftarrow v \bmod 2^s \end{aligned}$$

Step 2: Return

$$M(w, y) \times 2^{2s} + (M(w + x, y + z) - M(w, y) - M(x, z)) \times 2^s + M(x, z) \quad (5)$$

Suppose that multiplying two integers of maximum size n requires time $t(n)$ in recursive algorithms. Since $w + x$ and $y + z$ can be up to $1 + n/2$ digits, we have

$$t(n) \leq t(n/2) + t(n/2) + t(1 + n/2) + cn \quad (6)$$

for every $n \geq n_0$. This gives $t(n) \in O(n^{\log_2 3})$ [36].

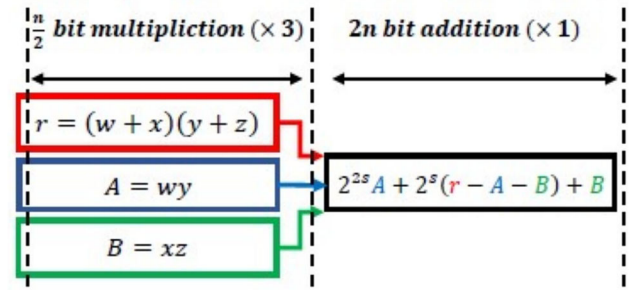


Fig. 7 One multiplication decomposition iteration

Figure 7 shows one iteration of an n -bit multiplication decomposition into three $n/2$ -bit multiplications, which act concurrently as they have no data dependencies followed by a $2n$ -bit addition. Each $n/2$ -bit multiplier produces an n -bit product (r , A , and B respectively for the multipliers from top to bottom). To be precise, it should be noted that as w , x , y , and z are of length $n/2$ bits, so $(w + x)$ and $(y + z)$ will result in $n/2 + 1$ bits, and in the cases that $n/2 + 1$ is odd, we should consider the multiplier to be $(n/2 + 2)$ -bit so that no problem occurs when halving the length in the next decomposition iteration. We have accurately considered that in our implementation and evaluations. However, to keep the figures simple and consistent with the idea of decomposing to half-length, we use $n/2$ -bit instead of $(n/2 + 1)$ -bit. After the parallel multiplications, there will be a set of additions that altogether result in a $2n$ -bit final output. As formula 4 suggests, the largest term of this addition would be $2^{2s}A$. Since A is an n -bit value and $2^{2s}A$ means shifting A $2s$ times to left (i.e., $2 \times n/2 = n$ times), so the addition will be of length $n + n = 2n$ bits as imposed by its largest operand.

Note that, as mentioned just before, the terms 2^{2s} and 2^s in formula 4 are just logical shifts to the left $2s$ and s (which are predefined constants) times respectively, and do not need any cryptographic operations during garbling and evaluation phases, and they are considered cost-free.

Although the addition is $2n$ -bit long, it does not need to be calculated by $2n$ -bit adders. Actually, it can be performed through n -bit additions. The description follows:

In the middle term of formula 4, we have $r - A - B = a - (A + B)$ which will be calculated as $r + 2^s(A + B) = r + 1^s(A + B) + 1$. As r , A , and B are n -bit operands, three n -bit additions will be required to sum up this part. Note that one's complement operation is done by inverting the bits, and inversion in the garbled circuits is done by XORing with 1 which is cost-free when FreeXOR optimization is used.

Then, when the n -bit result of $r - A - B$ is ready, it will be shifted to the left by s ($= n/2$) places (to obtain $2^s(r - A - B)$), resulting in an $3n/2$ -bit value with the first $n/2$ rightmost bits all zeros. As Fig. 8 shows, the addition of this value with B (the rightmost argument in formula 4) could be done using only an n -bit adder instead of an $3n/2$ bit one.

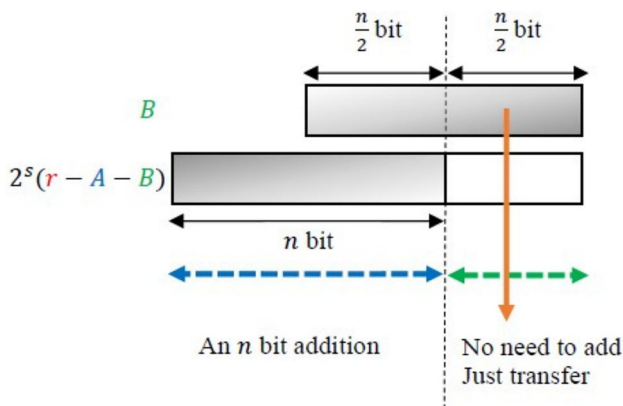


Fig. 8 Performing $3n/2$ bit additions in Formula 4 using an n -bit adder

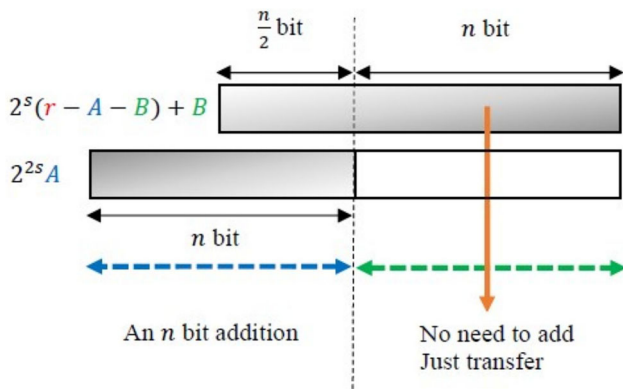


Fig. 9 Adding $3n/2$ bit and $2n$ -bit values in formula 4 using an n -bit adder

By the same reasoning, Fig. 9 shows that for summing up the $3n/2$ bit value $2^S(r - A - B) + B$ and $2n$ bit value $2^{2S}A$, only a single n -bit adder is sufficient.

So, the addition section of Fig. 7 will be performed by five consecutive n -bit addition operations. A one-iteration decomposition for a 512-bit multiplication will result in three parallel 256-bit multiplications and five consecutive 512-bit additions.

Depending on the level of parallelism the user demands for multiplications, each multiplication, in each iteration of decomposition could be divided again into concurrent instances with half the bit widths. This is formalized by the formula 5.

It shows the decomposition process of multiplication as a recursive notation. Each $M(...)$ represents one multiplication involved in the current step which could be decomposed again in the same way the previous larger multiplication (with twice the bit width) was decomposed into them. Figure 10 shows how an n -bit multiplication component would look like when the second phase of decomposition takes place on it.

In this stage, there will be nine parallel multiplications each $n/4$ -bit wide. Following each set of three multiplications

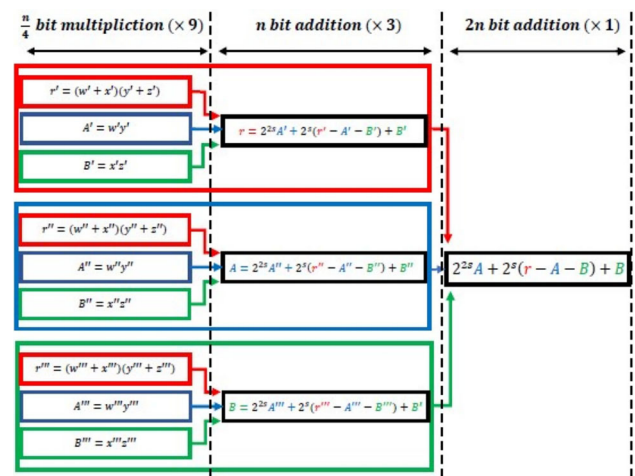


Fig. 10 Two steps of multiplication decomposition

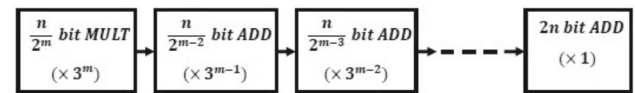


Fig. 11 Generalizing the number of concurrent multiplications and additions in the m_{th} step of Decomposition

is an addition process resulting in n -bit outputs (as mentioned before, this addition process could be performed through five $n/2$ -bit single additions). And, finally, one addition process sums up the results of the former three additions to produce the final $2n$ -bit product.

The parentheses beginning with $\times$, indicate how many concurrent operations of that type exist at that stage. Figure 11 generalizes the decomposition process for the m_{th} step. As the parentheses show, the set $\{3^m, 3^{m-1}, 3^{m-2}, \dots, 1\}$ defines the number of the concurrent operations specified by the set $\left\{ \frac{n}{2^m} \text{bit MULT}, \frac{n}{2^{m-2}} \text{bit ADD}, \frac{n}{2^{m-3}} \text{bit ADD}, \dots, 2n \text{bit ADD} \right\}$.

6 Structure of GCMult

Figure 12 depicts the various components of GCMult. The central processing unit consists of a series of concurrent Half-Gate ANDs, while the control section encompasses the distributor and collector. A set of RNGs is responsible for generating the required random labels for circuit wires. The system also incorporates multiple independent memory banks, which work together under the supervision of the distributor to store both internal and final output data. These data are then collected and transmitted to the host PC by the collector in the form of the garbled multiplication circuit.

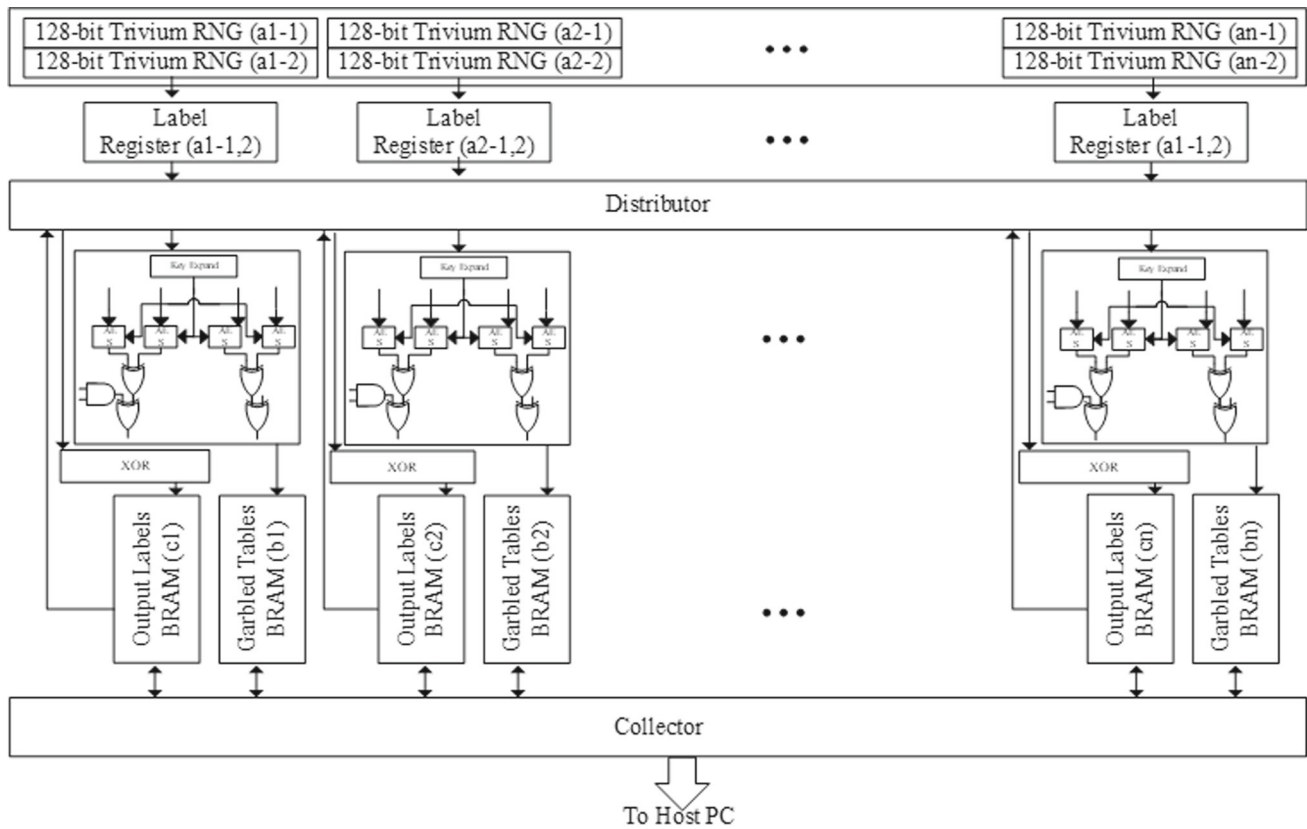


Fig. 12 GCMult structure

7 Label generator

The label generator comprises multiple 128-bit Random Number Generators (RNGs), each producing one random bit per cycle. To maximize concurrency, each garbling core is assigned a dedicated 256-bit RNG (128 bits for each input label of a gate), allowing all cores to garble their respective sections of the multiplier circuit simultaneously. We employ a pseudo-random number generator based on Trivium [37] to minimize FPGA resource usage while ensuring adequate security. Each set of TRNGs is activated only when new keys need to be generated.

8 Garbling engine

The GCMult system employs multiple independent and identical instances of Half-Gate AND components, determined by the decomposition iteration parameter. For example, with two decomposition iterations, there are nine Half-Gate AND units, each corresponding to a layer of decomposition performed by the Karatsuba algorithm (Section 5). Since each layer of the decomposed multiplication operates independently of the others with respect to input data, the gates

within each layer can be garbled in parallel. To maximize parallelism, we assign one XOR gate and a dedicated pair of random number generators (RNGs) to each layer.

The garbling engine is responsible for generating garbled tables and the output key corresponding to the logical value 0. This process is based on the two input keys associated with value 0 and the Boolean logic of the gate. Utilizing the free-XOR optimization, the key for the logical value 1 is derived by XORing the key for value 0 with a fixed global offset. Additionally, the garbling engine incorporates both row-reduction and Half-Gate optimizations, ensuring that each garbled table consists of only two rows per gate. XOR gates do not require garbled tables; instead, their output keys are computed directly using simple XOR operations.

To enhance computational efficiency, the garbling engine is equipped with four AES cores, which implement a 10-stage pipeline based on the AES design from [22, 38]. While a fully loaded pipeline could theoretically generate one garbled table per clock cycle, our design prioritizes parallelism while minimizing resource overhead. Consequently, rather than employing a pipelined approach, each AES core processes a non-XOR gate within 10 clock cycles, balancing performance with hardware resource constraints.

8.1 Internal memory architecture

The system employs three distinct categories of internal memory, each serving a specific function.

8.1.1 Label register memory

The first category, labeled as “Label Register” in Fig. 12, consists of separate dual-port BRAMs dedicated to storing the latest labels of the two input wires of each gate, as generated by the random number generators (RNGs). To ensure parallel access without conflicts, each Label Register is allocated to a separate BRAM corresponding to a specific garbling engine, such as a Half-Gate AND component or an XOR gate. This design allows independent and simultaneous access across multiple garbling engines.

8.1.2 Output label memory

The second category, referred to as “Output Labels” in Fig. 12, consists of BRAMs that store the output labels produced by Half-Gate ANDs and XOR gates. These stored outputs serve as inputs for subsequent gates in the computation and are accessed by the distributor to facilitate efficient data flow within the circuit.

8.1.3 Garbled table memory

The final category, “Garbled Tables”, stores the garbled tables generated by the Half-Gate ANDs in the decomposed Boolean circuit of the multiplication process. The contents of the Output Label Memory and Garbled Table Memory are later retrieved by the collector and transmitted to the host PC for further processing.

This structured memory organization enhances parallelism and minimizes access conflicts, optimizing the efficiency of the garbling process.

8.2 Distributor: control logic and circuit embedding

The distributor serves as the central control unit of the system, embedding the Boolean circuit representation of the decomposed multiplication directly within its architecture. Since GCMult is specifically designed as a multiplication accelerator, it does not require an externally provided netlist. Instead, all necessary circuit data is statically programmed into the distributor. This design choice offers some advantages, which will be discussed in Section 6.5.

In addition to circuit management, the distributor is responsible for issuing commands to the random number generators (RNGs) to generate new labels as needed. It operates in a SIMD-like manner when controlling multiple garbling cores and managing memory accesses in parallel. Based on

a predefined memory management strategy with respect to BRAM amount usage threshold, the distributor notifies the collector, which then transfers the processed data to the host PC.

8.3 Features specific to GCMult

As GCMult is an accelerator tailored specifically for garbling multiplication, the following features are specifically related to it.

General-purpose GC systems typically store the netlist in a BRAM using formats like HSCD. BRAMs introduce a one-cycle latency between receiving an input address (gate index) and outputting the corresponding data (input indices and gate logic). Before advancing to the next gate by incrementing the gate index, the control logic must first verify whether the input keys for the current gate are ready. This process results in a latency of two cycles per gate. However, our GC accelerator design avoids this issue, as it is specialized for a single function—multiplication—with its netlist directly embedded in the control logic.

Since the Boolean circuit is inherently embedded within the system, there is no requirement for an additional subsystem or associated memory to receive and store the netlist. This integration optimizes communication bandwidth, reduces hardware resource consumption, and enhances the efficiency of the garbling process.

The keys corresponding to gate outputs, whether generated by the garbling engine or through XOR operations, are stored in a dual-port BRAM. These keys are later accessed for subsequent gates that rely on the current gate’s outputs. In [23], BRAM access takes four cycles per gate, but this can be reduced to two cycles by using dual-port BRAM instead of single-port BRAM. In FASE [22], which adopts a similar approach to GCMult by using HG optimization for AND gates, two keys are read from memory, and one key is written back per gate, making it theoretically possible to process each gate in 1.5 cycles. However, in our design, the netlist is static and hardwired in the control logic, allowing the system to track the status of all circuit nodes in advance. This enables us to store each gate’s two input labels in separate dual-port BRAMs, allowing four inputs to be read simultaneously per gate. As a result, on average, we achieve a garbling time of just 0.75 cycles per XOR gate.

9 Evaluation

GCMULT is implemented on a Xilinx Virtex UltraScale VCU108 (XCVU095) FPGA. Although digital clock management resources (e.g., MMCM) can enable substantially higher operating frequencies, 200 MHz is the clock frequency used in our evaluation. We evaluated our design multiple times, varying two key parameters: the input size and the number of decomposition iterations. Our experiments were conducted using input sizes of 8, 16, 32, 64, 128, and 256 bits (i.e., 8×8 , 16×16 , ..., 256×256 multiplications). Additionally, we evaluated the proposed approach with one, two, and three decomposition iterations. In all comparisons between our work and related approaches, we focus on the case with two decomposition iterations ($di=2$). While this configuration is not performance-optimal, it constrains FPGA resource utilization to the levels reported in prior works, thereby enabling a fair and direct comparison with baseline implementations.

9.1 Resource utilization

The resource utilization on this platform is detailed in Table 2. Specifically, we present the resource consumption of three distinct subsystems: The Label Generator, the Control Unit (or Distributor), and the Garbling Engine, which consists of parallel Half-Gate ANDs and XORs. Notably, varying the number of decomposition iterations affects only the resource usage of the Label Generator and Garbling Engine, while the resource consumption of the Distributor remains constant.

9.2 Garbling time and number of gates

Our proposed approach utilizes the Karatsuba algorithm to improve efficiency by decomposing large multiplications into smaller, parallel multiplications and additions. This method significantly reduces the number of gates, particularly the high-cost AND gates. Table 3 demonstrates the effectiveness of this approach. As observed, for larger multiplications (32-bit and above), introducing an additional iteration of decomposition reduces the number of AND gates by approximately 50% on average while decreasing the garbling time to one-third.

9.3 Performance comparison with previous works

Table 4 presents a comparison of the garbling time (measured in clock cycles) for multiplication functions of varying sizes, along with the total number of AND and XOR gates in the Boolean circuit of GCMULT with two decomposition iterations ($di=2$). This comparison is made against two well-known FPGA-based GC accelerators [22, 23], which utilize multiple cores in different configurations. FASE [22]

employs a pipelined architecture with 10 stages, while [23] implements a multi-core design with 43 cores. In contrast, GCMULT, with $di=2$, features 9 garbling engine cores. To ensure a fair evaluation, we compare performance on a per-core basis. As observed in the table, GCMult exhibits slightly lower performance than FASE for small-scale multiplications (below 16-bit), though it still significantly outperforms the other well-known general-purpose accelerator presented in [23]. This performance gap in smaller multiplications is expected due to the overhead associated with GCMult's parallelized decomposition approach. However, as the multiplication size increases, GCMult demonstrates a substantial advantage over FASE, the fastest existing generic FPGA-based garbled circuit accelerator, when utilized for multiplication garbling.

For larger multiplications, GCMult efficiently utilizes its optimized architecture, leading to notable improvements in computation speed. Experimental results indicate that for 64-bit multiplications, GCMult achieves a 216% speedup compared to FASE. This performance gain becomes even more pronounced as the multiplication size increases, reaching nearly 400% improvement for 256-bit multiplications. These results highlight the scalability and efficiency of GCMult in handling complex and large-scale secure multiplications, making it a promising solution for privacy-preserving computations.

Both related works [22, 23] that we compared our approach with are general-purpose accelerators, whereas our design is specialized for multiplication. Additionally, the number of concurrent garbling cores implemented in these works differs from GCMult. Therefore, while a direct comparison of resource utilization may not be entirely fair, Table 5 provides a summary of the total FPGA resource consumption for FASE [22] and our design on the Xilinx Virtex UltraScale VCU108 (XCVU095) FPGA. This comparison offers a clearer perspective on the resource utilization of our approach.

9.4 Throughput enhancement compared to software approaches

Finally, we evaluate the performance of GCMULT against two software-based GC implementations: OblivM [39] and TinyGarble [40]. The software experiments are conducted on an Intel® Core™ i3-9100F processor running at 3.6 GHz with 64 GB of memory. Due to the significant difference in clock frequencies between the FPGA and the CPU, performance is measured in absolute time (microseconds) rather than clock cycles. As shown in Table 6, GCMULT achieves an average speedup of over $1426\times$ compared to OblivM and more than $45\times$ compared to the fastest software-based GC implementation, TinyGarble.

Table 4 Comparison of clock cycles required for garbling by GCMult with FASE [22] and accelerator in [23], also the number of (AND, XOR) gates

		[23]	FASE [22]	GCMult (di = 2)	Imp. Over [23]	Imp. Over FASE [22]
A*B (8)	Clock Cycle	4400	619	932	22.55 (1777%)	0.73 (-30%)
	# of XOR gates	352*	352	352	1	1
	# of AND gates	120	120	146	0.82	0.82
A*B (16)	Clock Cycle	13,000**	2546**	2131	29.14 (2436%)	1.32 (21%)
	# of XOR gates	1472	1472	1046	1.4	1.4
	# of AND gates	496	496	436	1.13	1.13
A*B (32)	Clock Cycle	36,000	10,500	5272	32.62 (2784%)	2.21 (110%)
	# of XOR gates	6016	6016	3298	1.82	1.82
	# of AND gates	2016	2016	1448	1.39	1.39
A*B (64)	Clock Cycle	110,000	42,800	14,530	36.17 (3139%)	3.27 (216%)
	# of XOR gates	24,320	24,320	11,258	2.16	2.16
	# of AND gates	8128	8128	5200	1.56	1.56
A*B (128)	Clock Cycle	367,000	172,972	44,950	39.00 (3423%)	4.27 (316%)
	# of XOR gates	97,920	97,920	41,002	2.38	2.38
	# of AND gates	32,640	32,640	19,616	1.66	1.66
A*B (256)	Clock Cycle	1,300,000	695,607	153,406	40.48 (3571%)	5.03 (392%)
	# of XOR gates	379,366	379,366	155,786	2.43	2.43
	# of AND gates	130,816	130,816	76,096	1.71	1.71

*Data are extracted from [22, 23] and slightly different with the value expected as number of XORs by Table 1

**Interpolated from the results provided in [22, 23] for 16-bit

Bold value indicates the process time (clock cycles) improvement achieved by our proposed approach compared with other works

9.5 Communication bandwidth comparison

If we define the following netlist parameters: M (total number of input bits), N (total number of output bits), and A (total number of AND gates), then the amount of data transferred from the FPGA to the host CPU for garbling a single netlist using FASE [22] and GCMult, which use Half-Gate optimization, are given by: $((M + 2A)K + N)$ bits (data transfer amount for XOR gates is neglectable). For the accelerator

proposed in [23], which does not incorporate half-gate optimization, the garbled tables consist of three rows instead of two. Consequently, the amount of data transferred is given by: $(3G + 3A)K$ bits. Table 7 presents a comparison of data transfer from the FPGA to the host PC for different multiplication sizes, highlighting our approach alongside two other accelerators [22, 23].

Table 5 Comparison of FPGA resource usage with FASE [22]

	FASE[22]		GCMult					
			Decomposition iterations					
			1		2		3	
	No	%	No	%	No	%	No	%
LUT	50,035	9.31	9917	1.84	28,325	5.27	83,549	15.54
FF	11,416	1.06	5995	0.55	13,207	1.22	34,843	3.23
BRAM	68.5	3.96	62.5	3.61	101.5	5.86	218.5	12.63
LUT RAM	596	0.74	250	0.32	592	0.77	1618	2.10

Bold value denotes resource utilization of our method when a specific decomposition iterations (=2) is chosen, compared with one of the related works

Table 6 Garbling time (microseconds) with two best SW approaches[39, 40]

	[39]	[40]	GCMult (di = 2)	Imp. Over ObliVM	Imp. Over TinyGarble
A*B (8)	1.03×10^4	49.7	5.9	1753.36	8.42
A*B (16)	1.69×10^4	188	13.48	1253.45	13.94
A*B (32)	3.79×10^4	772	32.96	1150.82	23.43
A*B (64)	1.1×10^5	3310	89.2	1237.05	37.11
A*B (128)	4.14×10^5	13,400	270.8	1528.04	49.66
A*B (256)	1.38×10^6	54,100	910.48	1514.93	59.46

Table 7 Comparison of communication bandwidth required by GCMult with FASE [22] and Accelerator presented in [23]

	[23] (3G + 3A)K (bits)	FASE [22] ((M + 2A)K + N) (bits)	GCMult (di = 2) ((M + 2A)K + N) (bits)	Imp. Over [23] (%)	Imp. Over FASE [22] (%)
A*B (8) M = 16, N = 16, K = 128	227,328	32,784	39,440	476.38	- 16.87
A*B (16) M = 32, N = 32, K = 128	1,291,776	246,304	115,744	1016.06	112.80
A*B (32) M = 64, N = 64, K = 128	3,858,432	524,352	378,944	918.20	38.37
A*B (64) M = 128, N = 128, K = 128	15,581,184	2,097,280	1,347,712	1056.12	55.61
A*B (128) M = 256, N = 256, K = 128	62,530,560	8,342,784	5,054,720	1137.07	65.04
A*B (256) M = 512, N = 512, K = 128	246,143,232	33,554,944	19,546,624	1159.26	71.66

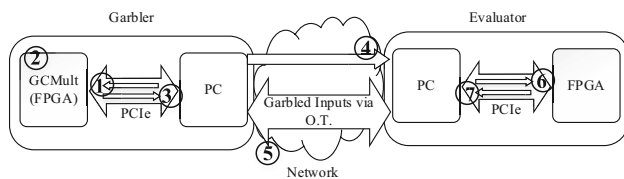


Fig. 13 Network configuration for the garbler and evaluator

9.6 End-to-end latency comparison

Because the scope of this work is limited to the garbling phase, we are unable to report an exact end-to-end latency for a complete secure two-party multiplication protocol. Nevertheless, using a representative network configuration for the garbler and evaluator (illustrated in Fig. 13), the garbling times provided in Table 4, and the communication parameters summarized in Table 8, we estimate a lower bound on the latency reduction achieved by the proposed method.

In Fig. 13, the steps of the end-to-end secure computation that are relevant to the comparison are numbered, and their descriptions are given below.

1. In conventional garbled-circuit frameworks, the Boolean circuit representation of the target function is transmitted to the garbler accelerator via a PCIe link. However, since GCMult is a multiplication-specific garbler, the Boolean specification is pre-defined within its controller. Consequently, no circuit description needs to be transferred from the host PC. This step therefore consists solely of issuing a request to the garbler accelerator to initiate a new garbling process. By eliminating circuit transmission, the proposed approach reduces the overall point-to-point secure multiplication latency by approximately BCS/PTR .

2. The garbling latency is reported in Table 4.

3. Upon completion of the garbling process, the generated garbled tables are transmitted back to the host PC via the PCIe interface. Considering only the data transfer latency, the delay of this step is approximately GTS/PTR , where GTS denotes the size of the garbled tables. The values of GTS are provided in Table 7.

4. Subsequently, the garbled tables are transferred from the garbler's host PC to the evaluator's host PC over the network. The corresponding communication delay is approximately GTS/NTR , where NTR represents the network transmission rate.

5. This stage involves the transfer of garbled input data between the garbler and evaluator through the Oblivious Transfer (OT) protocol. As this step is common to both the proposed architecture and prior approaches, it is excluded from the comparative performance analysis.

6. The garbled tables and garbled inputs are then delivered to the evaluator accelerator (which is beyond the scope of this work) via PCIe. Because the size of the garbled inputs is

Table 8 Parameters referred to in end-to-end latency comparison

Parameter	Par. variable name
Boolean circuit representation size	BCS
Garbling latency	GL
Garbled tables size (bits)	GTS
PCIe transfer rate	PTR
Network transfer rate	NTR

identical across the compared approaches, only the garbled table size is considered in the performance evaluation. The associated transfer latency is therefore again approximated as GTS/PTR .

10 7. Finally, the evaluation result is returned to the host system. As this operation is not specific to the proposed method, it is likewise excluded from the comparative analysis.

Excluding the steps that are identical in both the proposed architecture and prior designs, the end-to-end latency for earlier approaches—those that require transmitting the Boolean representation of the function to the garbler—can be expressed as $\frac{BCS}{PTR} + GL + 2 \times \frac{GTS}{PTR} + \frac{GTS}{NTR}$.

In the case of GCMult, the first term is eliminated because the Boolean specification is already embedded in the garbler. Moreover, since BCS is significantly smaller than PTR, we omit this term in our numerical evaluations to simplify the comparative analysis. Table 9 summarizes the comparison. Note that in our calculations, the PTR is assumed to be 63 Gb/s for PCIe $\times$ 8 Gen 3, and the NTR is taken to be 10 Mb/s (10BASE-T standard).

10.1 Energy efficiency analysis

Since generating garbled circuits involves a large number of cryptographic operations, the energy consumption of the proposed hardware accelerator was evaluated to determine its suitability for high-throughput and energy-constrained environments.

Table 10 summarizes the estimated energy consumption of the proposed architecture. Power analysis was performed using the vendor-provided power estimation tool, Xilinx Power Analyzer (XPA), based on the post-implementation netlist. The design was synthesized and implemented operating at a clock frequency of 200 MHz.

Direct energy efficiency comparisons with prior works on hardware accelerators for Yao's Garbled Circuits are not possible, as energy metrics are typically unreported. Our detailed

Table 9 End-to-end latency (microseconds) comparison with those of FASE [22] and accelerator in [23]

	[23]	FASE [22]	GCMult (di = 2)	Imp. Over [23]	Imp. Over FASE [22]
A*B (8)	22,767	3283	3951	27.53 (2275%)	0.92 (-18%)
A*B (16)	129,300	24,654	11,591	53.30 (4851%)	2.36 (125%)
A*B (32)	386,193	52,518	37,939	48.63 (4385%)	1.53 (42%)
A*B (64)	1,559,309	210,065	134,905	55.22 (5044%)	1.73 (62%)
A*B (128)	6,257,364	835,638	505,917	59.09 (5431%)	1.83 (72%)
A*B (256)	24,630,365	3,360,962	1,956,254	60.15 (5537%)	1.90 (80%)

Table 10 Energy analysis across varying input sizes and decomposition iterations in proposed approach

	A*B (8)			A*B (16)			A*B (32)			A*B (64)			A*B (128)			A*B (256)		
Iteration No.	1	2	3	1	2	3	1	2	3	1	2	3	1	2	3	1	2	3
Core No.	3	9	27	3	9	27	3	9	27	3	9	27	3	9	27	3	9	27
Energy (μ J)	4.13	11.5	38.0	12.2	26.3	79.4	40.4	65.0	169.6	144.1	179.3	379.2	541.5	537.6	836.9	2095.6	1835.0	2244.4

Table 11 Energy (μ J) consumption by GCMult and identical acc. w.t. decomposition

	Identical acc. without decomposition (μ J)	GCMult (di = 2) (μ J)	Improvement
A*B (8)	8.2	11.5	0.71 (− 28%)
A*B (16)	33.8	26.3	1.28 (28%)
A*B (32)	139.5	65.0	2.14 (114%)
A*B (64)	568.8	179.3	3.17 (217%)
A*B (128)	2298.9	537.6	4.27 (327%)
A*B (256)	9245.4	1835.0	5.03 (403%)

energy analysis across varying input sizes and Karatsuba decomposition levels establishes a clear baseline for future studies and demonstrates the energy benefits of our approach. Additionally, Table 11 presents the energy consumption of a baseline accelerator identical to our architecture except for the omission of Karatsuba decomposition and the adoption

of a 10-stage pipelined AES encryptor, closely aligning with the conventional methodology found in FASE [20].

10.2 System integration and applications

Primitive Instantiation Architecture. The accelerator targets high-precision secure multiplication in practical two-party computation deployments. Designed as a composable primitive, GCMult can be instantiated within the garbled circuit of any function requiring large-integer products. Note that many applications—most notably machine learning algorithms—quantize real-valued inputs to high-precision integers prior to computation, subsequently rescaling the outputs back to the real domain. Via compiler directives (e.g., pragmas or hardware-description flags), developers can guide the garbler to selectively instantiate GCMult for high-precision operations while retaining legacy or custom multipliers for smaller bit-widths, ensuring the final garbled circuit employs the optimal primitive for each multiplication node.

Hybrid Integration Architecture. The FPGA operates as a garbling oracle within existing MPC stacks (ABY [41], EMP-toolkit [42], MP-SPDZ [43]). A lightweight Hardware Abstraction Layer exposes the multiplier as a standard Boolean circuit primitive; the host CPU manages protocol

orchestration (OT, network I/O) while streaming large-multiplication sub-circuits to the FPGA. This separation allows hybrid-protocol compilers to substitute software-based AND gates with pipelined Half-Gate evaluations, seamlessly accelerating any garbled function regardless of its high-level logic. The design supports both cloud FPGA instances (AWS F1 [44]) for cross-organizational analytics and resource-constrained edge deployments (Xilinx Zynq [45]) for latency-sensitive automotive or IoT applications.

In real-world deployments, the proposed accelerator enables practical secure computation for privacy-sensitive domains requiring high-precision arithmetic, such as cross-institutional financial analytics (e.g., collaborative risk modeling and sealed-bid auctions involving 128-bit monetary values), privacy-preserving machine learning (e.g., secure training of medical diagnostic models where two hospitals jointly compute fixed-point gradient updates on 64–256-bit weights without exposing patient data). These applications are characterized by deep multiplicative circuits—whether in iterative optimization algorithms or high-degree polynomial evaluations—where the Karatsuba decomposition yields compounding reductions in garbled-circuit size while simultaneously accelerating garbling through increased parallelism, thereby transforming computationally prohibitive protocols into deployable solutions for regulated industries subject to GDPR [46] or HIPAA [47] constraints.

11 Conclusion

In this paper, we introduced GCMult, an FPGA-based accelerator specifically designed for secure two-party multiplication using Yao's Garbled Circuits (GC) protocol. Our approach leverages a parallelized decomposition technique for Boolean circuits, incorporating the Karatsuba algorithm to enhance concurrency while significantly reducing the number of gates, particularly the high-cost AND gates. By exploiting the inherent parallelism in multiplication, GCMult achieves substantial improvements in both computational efficiency and communication overhead.

Experimental evaluations on FPGA demonstrate that GCMult reduces garbling time by an average of 258% and improves communication bandwidth by more than 60% on average compared to the best existing hardware-based accelerator. Additionally, it delivers an impressive 4140% speedup over the fastest known software implementation, reinforcing its potential for scalable and high-performance privacy-preserving computations.

These results establish GCMult as a practical and efficient solution for secure multiplications, paving the way for further optimizations in secure computation frameworks. Future research can explore extending this approach to other fundamental arithmetic operations and integrating it into

broader cryptographic protocols to further advance privacy-preserving computing.

Author contributions All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Hiva Assasi and Shahram Etemadi Borujeni.

Funding Not applicable.

Data availability No datasets were generated or analysed during the current study.

Declarations

Ethical approval Not applicable.

Consent for publication We hereby provide consent for the publication of the manuscript detailed above, including any accompanying images or data contained within the manuscript that may directly or indirectly disclose our identity.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Wang, Y., Li, T., Qin, H., Li, J., Gao, W., Liu, Z., Xu, Q.: A brief survey on secure multi-party computing in the presence of rational parties. *J. Ambient. Intell. Humaniz. Comput.* **6**, 807–824 (2015)
2. Harn, L., Xia, Z., Hsu, C.: Non-interactive secure multi-party arithmetic computations with confidentiality for P2P networks. *Peer-to-Peer Netw. Appl.* **14**, 722–728 (2021)
3. Yao, A.C.-C.: *How to generate and exchange secrets*. in *27th annual symposium on foundations of computer science (Sfcs 1986)*. 1986. IEEE.
4. Alam, M., Emmanuel, N., Khan, T., Xiang, Y., Hassan, H.: Garbled role-based access control in the cloud. *J. Ambient. Intell. Humaniz. Comput.* **9**, 1153–1166 (2018)
5. Zhu, Y., Li, X.: Privacy-preserving k-means clustering with local synchronization in peer-to-peer networks. *Peer-to-Peer Netw. Appl.* **13**(6), 2272–2284 (2020)
6. Haque, A., Heath, D., Kolesnikov, V., Lu, S., Ostrovsky, R. and Shah, A., 2022, May. Garbled circuits with sublinear evaluator. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques* (pp. 37–64). Cham: Springer International Publishing.
7. Zhao, Q., Liu, X., Xu, H., Li, Y.: Practical reusable garbled circuits with parallel updates. *Comput. Stand. Interfaces* **86**, 103721 (2023)
8. Cao, Z., C. Huang, and Y. Li. A Study on the improvement of Computation, Communication and Security in Garbled Circuits. in

- 2021 6th International Conference on Intelligent Computing and Signal Processing (ICSP). 2021. IEEE.
9. Ashur, T., Cohen, E., Hazay, C., Yanai, A.: A new framework for garbled circuits. IACR Cryptol. ePrint Arch. **2021**, 739 (2021)
 10. Beaver, D., Micali, S., Rogaway, P.: The round complexity of secure protocols. In: *Proceedings of the Twenty-second Annual ACM Symposium on Theory of Computing*. pp. 503–513 (1990).
 11. Malkhi, D., Nisan, N., Pinkas, B. and Sella, Y., *Fairplay-secure two-party computation system*. In: *USENIX Security Symposium*. vol. 4, p. 9 (2004)
 12. Naor, M., Pinkas, B., Sumner, R.: Privacy preserving auctions and mechanism design. *Proceedings of the 1st ACM Conference on Electronic Commerce*, . pp. 129–139 (1999).
 13. Kolesnikov, V. and Schneider, T., 2008. Improved garbled circuit: Free XOR gates and applications. In *Automata, Languages and Programming: 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7–11, 2008, Proceedings, Part II* 35 (pp. 486–498). Springer Berlin Heidelberg
 14. Pinkas, B., Schneider, T., Smart, N.P. and Williams, S.C., 2009. Secure two-party computation is practical. In *Advances in Cryptology—ASIACRYPT 2009: 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6–10, 2009. Proceedings 15* (pp. 250–267). Springer Berlin Heidelberg.
 15. Innocent, A.A.T., K, S., Prakash, G.: Universal gates on garbled circuit construction. *Concurrency Comput. Pract. Exp.* **31**(17), e5236 (2019)
 16. Buescher, N. and S. Katzenbeisser. Faster secure +computation through automatic parallelization. in *24th USENIX Security Symposium (USENIX Security 15)*. 2015.
 17. Z Zhang, Y., Steele, A. and Blanton, M., 2013, November. PICCO: a general-purpose compiler for private distributed computation. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security* (pp. 813–826).
 18. Barni, M., Bernaschi, M., Lazzeretti, R., Pignata, T. and Sabellico, A., 2013, September. Parallel implementation of GC-based MPC protocols in the semi-honest setting. In *International Workshop on Data Privacy Management* (pp. 66–82). Berlin, Heidelberg: Springer Berlin Heidelberg.
 19. Kartik N., Xiao S. W., Stratis I., Udi W., Nina T. and Elaine S., Graphsc: Parallel secure computation made easy. in *2015 IEEE symposium on security and privacy*. 2015. IEEE.
 20. Frederiksen, T.K., Jakobsen, T.P. and Nielsen, J.B., 2014. Faster maliciously secure two-party computation using the GPU. In *Security and Cryptography for Networks: 9th International Conference, SCN 2014, Amalfi, Italy, September 3–5, 2014. Proceedings 9* (pp. 358–379). Springer International Publishing.
 21. Songhori, E.M., Zeitouni, S., Dessouky, G., Schneider, T., Sadeghi, A.R. and Koushanfar, F., 2016, June. Garbledcpu: a mips processor for secure computation in hardware. In *Proceedings of the 53rd Annual Design Automation Conference* (pp. 1–6).
 22. Hussain, S.U. and Koushanfar, F., 2019, April. FASE: FPGA acceleration of secure function evaluation. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (pp. 280–288). IEEE.
 23. Fang, X., Ioannidis, S. and Leeser, M., 2017, February. Secure function evaluation using an fpga overlay architecture. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (pp. 257–266).
 24. Manuel, S.: Classification and generation of disturbance vectors for collision attacks against SHA-1. *Des. Codes Cryptogr.* **59**(1), 247–263 (2011)
 25. Wang, X., Yin, Y.L. and Yu, H., 2005. Finding collisions in the full SHA-1. In *Advances in Cryptology—CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14–18, 2005. Proceedings 25* (pp. 17–36). Springer Berlin Heidelberg
 26. Satoh, A., 2005, September. Hardware architecture and cost estimates for breaking SHA-1. In *International Conference on Information Security* (pp. 259–273). Berlin, Heidelberg: Springer Berlin Heidelberg.
 27. Mo, J., Gopinath, J. and Reagen, B., 2023, June. Haac: A hardware-software co-design to accelerate garbled circuits. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (pp. 1–13).
 28. Yao, A.C. *Protocols for secure computations*. In *23rd annual symposium on foundations of computer science (sfcs 1982)*. 1982. IEEE.
 29. Navarro, Ignacio. *On garbled circuits*. (2018).
 30. Rabin, M.O., 2005. How to exchange secrets with oblivious transfer. *Cryptology ePrint Archive*.
 31. Zahur, Samee, Mike Rosulek, and David Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. *Advances in Cryptology-EUROCRYPT 2015: 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26–30, 2015, Proceedings, Part II* 34. Springer Berlin Heidelberg, 2015.
 32. Parhami, B., 2010. *Computer arithmetic* (Vol. 20, No. 00). New York, NY: Oxford university press.
 33. Karatsuba, Anatolii. *Multiplication of multidigit numbers on automata*. Soviet physics doklady. Vol. 7. 1963.
 34. Pogue, T.E., Nicolici, N.: Karatsuba matrix multiplication and its efficient custom hardware implementations. *IEEE Trans. Comput.* (2025). <https://doi.org/10.1109/TC.2025.3525606>
 35. Das, M., Jajodia, B.: Hybrid recursive Karatsuba multiplications on FPGAs. *IEEE Embed. Syst. Lett.* (2025). <https://doi.org/10.1109/LES.2025.3538470>
 36. Pei, D., Salomaa, A., Ding, C.: *Chinese Remainder Theorem: Applications in Computing, Coding, Cryptography*. World Scientific (1996)
 37. De Cannière, C., 2006, August. Trivium: A stream cipher construction inspired by block cipher design principles. In *International Conference on Information Security* (pp. 171–186). Berlin, Heidelberg: Springer Berlin Heidelberg.
 38. S. U. Hussain. FASE: FPGA Acceleration of Secure Function Evaluation. [Online]. Available: <https://github.com/siamumar/FASE>
 39. Liu, C., Wang, X.S., Nayak, K., Huang, Y. and Shi, E., 2015, May. Oblivm: A programming framework for secure computation. In *2015 IEEE Symposium on Security and Privacy* (pp. 359–376). IEEE.
 40. Songhori, E.M., Hussain, S.U., Sadeghi, A.R., Schneider, T. and Koushanfar, F., 2015, May. Tinygarble: Highly compressed and scalable sequential garbled circuits. In *2015 IEEE Symposium on Security and Privacy* (pp. 411–428). IEEE.
 41. Demmler, D., Schneider, T. and Zohner, M., 2015, February. ABY-A framework for efficient mixed-protocol secure two-party computation. In *Ndss*.
 42. Wang, X., Malozemoff, A.J. and Katz, J., 2016. EMP-toolkit: Efficient MultiParty computation toolkit [online].
 43. Keller, M., 2020, October. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security* (pp. 1575–1590).
 44. Amazon Web Services, “Amazon EC2 F1 Instances,” *Amazon Web Services Documentation*, 2024.
 45. Xilinx Inc., “Zynq-7000 SoC Technical Reference Manual,” *UG585, Xilinx*, 2023.
 46. European Parliament and Council of the European Union, “General Data Protection Regulation (GDPR),” *Regulation (EU) 2016/679*, Apr. 2016.

47. U.S. Congress, “Health Insurance Portability and Accountability Act of 1996 (HIPAA),” *Public Law 104–191*, Aug. 21, 1996.

Publisher’s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.